



**MUSEO MALVINAS
ANTÁRTIDA Y
ATLÁNTICO SUR**

BARILOCHE · RÍO NEGRO

Sistema de Streaming Antártico

Manual del Desarrollador y Guía de Contribución

Equipo de Arquitectura y Desarrollo

*Jonás Porro
Galo Ignacio Orellana
Gaspar Corrarello*

Institución

*Universidad Nacional de Río Negro
Ingeniería en Computación
Ingeniería de Software I (B6021)*

Versión del Documento: 1.2

Fecha: 23-Junio-2026

Índice general

Historial de Revisiones	2
1. Introducción y Arquitectura	3
1.1. Arquitectura del Repositorio	3
2. Entorno Local de Desarrollo	4
2.1. Levantando el Manager (Backend)	4
2.2. Levantando el Player (Frontend)	4
3. Reglas y Estándares de Código	5
3.1. Laravel (Manager)	5
3.2. Svelte y Frontend (Player)	5
4. Flujo de Trabajo Git	6
4.1. Estrategia de Ramas	6
4.1.1. Estructura de Ramas	6
4.1.2. Pasos del Flujo	6
5. Integración y Despliegue Continuo (CI/CD)	8
5.1. Flujo de CI/CD	8
5.2. Cómo publicar una nueva versión	8
5.3. Despliegue Continuo (CD) con Portainer y GitOps	9

Historial de Revisiones

Este documento está sujeto a control de versiones. A continuación se detallan los cambios más significativos realizados a lo largo del tiempo.

Versión	Fecha	Descripción de Cambios	Autor / Revisor
1.0	18-Junio-2026	Creación del Manual del Desarrollador y Guía de Contribución	Gaspar Corrarello
1.1	18-Junio-2026	Actualización del flujo Git (main - develop - features) y CD.	Gaspar Corrarello
1.2	23-Junio-2026	Documentación de pruebas automatizadas con Vitest en Player.	Gaspar Corrarello

Capítulo 1

Introducción y Arquitectura

¡Bienvenido al equipo de desarrollo del Sistema de Streaming Antártico! Este manual está diseñado para proporcionarte todo el conocimiento técnico necesario para comenzar a programar de manera eficiente y siguiendo los estándares del proyecto.

1.1 Arquitectura del Repositorio

El proyecto está diseñado de forma modular. Dentro de la raíz del repositorio principal, encontrarás las siguientes carpetas clave:

- **manager:** Aplicación backend principal desarrollada en Laravel. Se encarga del Panel de Control, administración de monitores, base de datos y envío de eventos.
- **player:** Aplicación frontend desarrollada en Svelte. Se ejecuta en las pantallas del museo y recibe eventos en tiempo real para reproducir contenido.
- **redist:** Scripts de orquestación (Docker Compose standalone, Nginx Proxy Manager) utilizados para el despliegue en producción.
- **documentos:** Documentación formal en LaTeX y Markdown.

Capítulo 2

Entorno Local de Desarrollo

Para poder contribuir al código, necesitarás levantar los distintos módulos en tu computadora local.

2.1 Levantando el Manager (Backend)

El Manager está escrito en PHP 8.5 con el framework Laravel 13.

1. Ingresa a la carpeta del manager: `cd manager`.
2. Instala las dependencias de PHP: `composer install`.
3. Instala las dependencias de Frontend (si las hay): `npm install` o `pnpm install`.
4. Copia el archivo de entorno: `cp .env.example .env`.
5. Genera la clave de aplicación: `php artisan key:generate`.
6. Levanta la base de datos (puedes usar Laravel Sail o una instancia local de PostgreSQL/SQLite).
7. Ejecuta las migraciones: `php artisan migrate --seed`.
8. Inicia el servidor de desarrollo: `php artisan serve`.

2.2 Levantando el Player (Frontend)

El Player está construido con Svelte, Vite y Tailwind CSS v4.

1. Ingresa a la carpeta del player: `cd player`.
2. Instala las dependencias de Node: `pnpm install`.
3. Inicia el servidor de desarrollo de Vite: `pnpm run dev`.
4. Accede a la URL indicada (generalmente `http://localhost:5173`).
5. *Opcional:* Ejecuta las pruebas automatizadas (Vitest) con `pnpm run test`.

Capítulo 3

Reglas y Estándares de Código

3.1 Laravel (Manager)

El proyecto adopta los lineamientos de **Laravel Boost**. Algunas reglas fundamentales son:

- Utiliza constructores promocionados de PHP 8 para inyección de dependencias.
- Aplica un tipado estricto (*strict typing*) y utiliza *type hints* para todos los parámetros y retornos de métodos.
- Evita la creación directa de modelos en pruebas, prefiere usar Factories.
- Utiliza las convenciones de nomenclatura estándar de Laravel (Controladores en SingularController, tablas en plural, etc.).
- Pasa las pruebas automatizadas con PHPUnit antes de commitear.

3.2 Svelte y Frontend (Player)

- **TypeScript:** Utiliza TypeScript estricto. Define interfaces y tipos para la comunicación con las APIs.
- **Componentes:** Divide la lógica en componentes pequeños, encapsulando sus estilos y scripts localmente en el archivo `.svelte`.
- **Estilos:** Se utiliza Tailwind CSS v4. Evita escribir CSS plano a menos que sea estrictamente necesario o para animaciones muy específicas.
- **Testing:** Pasa las pruebas automatizadas con Vitest (`pnpm run test`) antes de commitear cambios importantes en el Frontend.

Capítulo 4

Flujo de Trabajo Git

4.1 Estrategia de Ramas

Hemos decidido estandarizar el ciclo de vida del código usando un flujo basado en **main, develop y feature branches**. Este modelo permite mantener un entorno de desarrollo activo y uno de producción siempre estable.

4.1.1 Estructura de Ramas

- **main:** Contiene el código estable de producción. Únicamente recibe uniones (*merges*) desde `develop` al realizar lanzamientos (*releases*).
- **develop:** Es la rama base para el trabajo diario. Contiene las últimas características probadas para la próxima versión.
- **feature/* y fix/*:** Ramas temporales creadas para el desarrollo de nuevas funciones o resolución de bugs. Nacen desde `develop` y se unen (*merge*) nuevamente hacia `develop`.

4.1.2 Pasos del Flujo

1. **Sincroniza tu entorno:** Siempre asegúrate de tener la última versión de la rama base antes de iniciar un trabajo: `git checkout develop` y luego `git pull origin develop`.
2. **Crea tu rama:** Crea una rama específica para tu tarea partiendo de `develop`: `git checkout -b feature/nuevo-reproductor` o `git checkout -b fix/error-autenticacion`.
3. **Commitea regularmente:** Haz commits atómicos (pequeños y lógicos). Los mensajes deben ser descriptivos y en tiempo verbal imperativo (ej. "Agrega validación de login").
4. **Abre un Pull Request (PR):** Una vez finalizado y probado localmente, empuja tu rama y abre un PR apuntando a la rama `develop`.
5. **Revisión de Código:** Etiqueta a un compañero para que revise tu PR. Aborda cualquier comentario de forma asíncrona.
6. **Merge y Limpieza:** Tras la aprobación, el PR se fusiona (*Merge*) hacia `develop` y la rama temporal se elimina para mantener limpio el repositorio.

Nota sobre Herramientas:

Te recomendamos utilizar **GitHub Desktop** para gestionar este flujo de trabajo. Su interfaz visual facilita enormemente la creación de ramas desde `develop`, la redacción de commits atómicos y la sincronización general, evitando los errores más comunes de la consola.

Capítulo 5

Integración y Despliegue Continuo (CI/CD)

El proyecto automatiza el ciclo de construcción y publicación de imágenes mediante **GitHub Actions**.

5.1 Flujo de CI/CD

Tanto el repositorio del `manager` como del `player` contienen flujos de trabajo (*workflows*) en la carpeta `.github/workflows/docker-image.yml`.

El comportamiento estandarizado es el siguiente:

- **Disparador (Trigger):** El proceso se inicia únicamente cuando se sube un nuevo *Tag* de Git con el formato `v*` (por ejemplo, `v1.0.0`).
- **Extracción de Versión:** El flujo extrae la versión del *Tag* (ej. `1.0.0`) y la inyecta temporalmente en la compilación (por ejemplo, en `public/version.txt`).
- **Construcción Docker:** Se utiliza Docker Buildx para ensamblar las imágenes optimizadas en caché.
- **Publicación en GHCR:** La imagen resultante se sube automáticamente al GitHub Container Registry (`ghcr.io`) etiquetada con la versión específica y la etiqueta `latest`.

5.2 Cómo publicar una nueva versión

Cuando el código en la rama `develop` está estable y se considera listo para producción:

1. Se aprueba y fusiona un Pull Request desde `develop` hacia `main`.
2. En la rama `main`, etiqueta el último commit localmente: `git tag v1.0.0` (o utiliza la interfaz gráfica).
3. Sube la etiqueta al repositorio remoto: `git push origin v1.0.0`
4. Dirígete a la pestaña *Actions* en GitHub para monitorear el progreso de la construcción.
5. Una vez terminada, la nueva imagen estará alojada y lista en `ghcr.io`.

Recomendación: Uso de GitHub Desktop

Para simplificar la creación de ramas y especialmente la gestión de *Tags* para los despliegues, recomendamos encarecidamente utilizar **GitHub Desktop**. Esta herramienta provee una interfaz visual que hace que agregar etiquetas a los commits y sincronizarlas con el servidor remoto sea un proceso intuitivo, rápido y libre de errores en consola.

5.3 Despliegue Continuo (CD) con Portainer y GitOps

La infraestructura de producción del Museo está configurada para consumir estas nuevas imágenes automáticamente sin intervención manual, cerrando el ciclo de Integración Continua (CI) con un Despliegue Continuo (CD) efectivo.

Esto se logra mediante la funcionalidad nativa de GitOps de Portainer (usando el método de Stack de Repositorio):

- **Sondeo (Polling):** Portainer consulta periódicamente (cada 5 minutos) el repositorio en GitHub para verificar el *hash* de la imagen `latest` o del archivo `docker-compose.yml`.
- **Detección y Descarga:** Al detectar que GitHub Actions terminó de publicar una nueva imagen en `ghcr.io`, Portainer inicia una descarga en segundo plano (*pull*).
- **Redespliegue Automático:** Una vez descargada, destruye los contenedores antiguos y levanta los nuevos de forma automática, garantizando que el sistema esté siempre sincronizado con el último release oficial de la rama `main`.